It’s not quite that simple, though; things never are. Even if a particular memory location is only ever accessed by one thread, you can *still* get cache ping-pong due to an effect known as *false sharing*.

8.2.3. False sharing

Processor caches don’t generally deal in individual memory locations; instead, they deal in blocks of memory called *cache lines*. These blocks of memory are typically 32 or 64 bytes in size, but the exact details depend on the particular processor model being used. Because the cache hardware only deals in cache-line-sized blocks of memory, small data items in adjacent memory locations will be in the same cache line. Sometimes this is good: if a set of data accessed by a thread is in the same cache line, this is better for the performance of the application than if the same set of data was spread over multiple cache lines. However, if the data items in a cache line are unrelated and need to be accessed by different threads, this can be a major cause of performance problems.

Suppose you have an array of `int` values and a set of threads that each access their own entry in the array but do so repeatedly, including updates. Since an `int` is typically much smaller than a cache line, quite a few of those array entries will be in the same cache line. Consequently, even though each thread only accesses its own array entry, the cache hardware *still* has to play cache ping-pong. Every time the thread accessing entry 0 needs to update the value, ownership of the cache line needs to be transferred to the processor running that thread, only to be transferred to the cache for the processor running the thread for entry 1 when that thread needs to update its data item. The cache line is shared, even though none of the data is, hence the term *false sharing*. The solution here is to structure the data so that data items to be accessed by the same thread are close together in memory (and thus more likely to be in the same cache line), whereas those that are to be accessed by separate threads are far apart in memory and thus more likely to be in separate cache lines. You’ll see how this affects the design of the code and data later in this chapter.